



Department of Artificial Intelligence and Machine Learning

Experiment No.: 4

To implement Red-Black Tree insertion.

Aim

To implement a **Red-Black Tree** and perform insertion, searching, and traversal operations while maintaining the self-balancing properties of the tree.

Learning Objectives

After completing this experiment, students will be able to:

- Understand the structure and properties of a Red-Black Tree.
- Implement insertion in a Red-Black Tree.
- Understand the role of node coloring in maintaining balance.
- Apply left and right rotations to balance the tree.
- Perform searching and traversal operations.

Theory

A **Red-Black Tree** is a self-balancing Binary Search Tree (BST) in which every node contains an additional attribute called **color**. The color of a node can either be **Red** or **Black**. The coloring rules ensure that the tree remains approximately balanced after insertion and deletion operations. This prevents the tree from becoming highly skewed and allows efficient tree operations.

A Red-Black Tree satisfies the following properties:

1. Every node is either **Red** or **Black**.
2. The root node is always **Black**.
3. Every NULL or NIL leaf node is considered **Black**.
4. If a node is **Red**, both of its children must be **Black**. Therefore, two Red nodes cannot appear consecutively.
5. Every path from a node to its descendant NIL nodes contains the same number of Black nodes.

During insertion, these properties may temporarily be violated. The tree is restored using **recoloring**, **left rotation**, and **right rotation** operations.



Algorithm

Red-Black Tree Insertion

1. Create a new node and assign its color as **Red**.
2. Insert the new node using the normal Binary Search Tree insertion procedure.
3. If the inserted node becomes the root, change its color to **Black**.
4. If the parent of the inserted node is Black, no further correction is required.
5. If both the inserted node and its parent are Red, identify the grandparent and uncle of the inserted node.
6. If the uncle is Red:
 - Change the parent color to Black.
 - Change the uncle color to Black.
 - Change the grandparent color to Red.
 - Continue checking from the grandparent.
7. If the uncle is Black or NIL, perform the appropriate left or right rotation.
8. Recolor the nodes after rotation.
9. Ensure that the root node remains Black.

Searching

1. Start from the root node.
2. Compare the search key with the current node.
3. If the key is equal to the current node, return the node.
4. If the key is smaller, move to the left subtree.
5. If the key is greater, move to the right subtree.
6. Repeat until the key is found or a NULL node is reached.



Sample Output

```
1 Inorder Traversal :
2 5 RED
3 10 BLACK
4 15 RED
5 20 BLACK
6 25 RED
7 30 BLACK
8 Element 15 found in the tree.
```

Problem Statement

2379. Minimum Recolors to Get K Consecutive Black Blocks

Easy

Topics

Companies

Hint

You are given a **0-indexed** string `blocks` of length `n`, where `blocks[i]` is either `'W'` or `'B'`, representing the color of the `ith` block. The characters `'W'` and `'B'` denote the colors white and black, respectively.

You are also given an integer `k`, which is the desired number of **consecutive** black blocks.

In one operation, you can **recolor** a white block such that it becomes a black block.

Return the **minimum** number of operations needed such that there is at least **one** occurrence of `k` consecutive black blocks.

Example 1:

Input: `blocks = "WBBWBBWBW"`, `k = 7`

Output: 3

Explanation:

One way to achieve 7 consecutive black blocks is to recolor the 0th, 3rd, and 4th blocks

so that `blocks = "BBBBBBWBW"`.

It can be shown that there is no way to achieve 7 consecutive black blocks in less than 3 operations.

Therefore, we return 3.

684. Redundant Connection

Medium

Topics

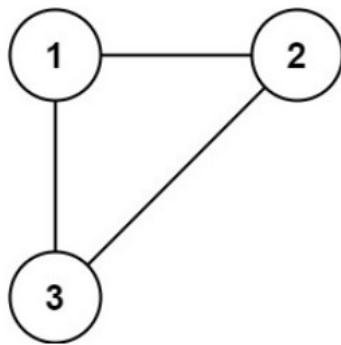
Companies

In this problem, a tree is an **undirected graph** that is connected and has no cycles.

You are given a graph that started as a tree with n nodes labeled from 1 to n , with one additional edge added. The added edge has two **different** vertices chosen from 1 to n , and was not an edge that already existed. The graph is represented as an array `edges` of length n where `edges[i] = [ai, bi]` indicates that there is an edge between nodes `ai` and `bi` in the graph.

Return an edge that can be removed so that the resulting graph is a tree of n nodes. If there are multiple answers, return the answer that occurs last in the input.

Example 1:



Input: `edges = [[1,2], [1,3], [2,3]]`
Output: `[2,3]`

Result

Thus, the **Red-Black Tree** was successfully implemented using Python. Insertion, searching, and inorder traversal operations were performed while maintaining the balancing properties of the Red-Black Tree.

Post-Lab Questions

1. What is the purpose of using colors in a Red-Black Tree?
2. List the five properties that must be satisfied by a Red-Black Tree.
3. Why is a newly inserted node initially colored **Red** instead of Black?
4. What violation can occur when a Red node is inserted under another Red node?
5. Explain the role of the **uncle node** during insertion and balancing.
6. What actions are performed when both the parent and uncle of the newly inserted node are Red?
7. Differentiate between **left rotation** and **right rotation** in a Red-Black Tree.



-
8. What happens when the uncle node is Black or NIL during insertion?
 9. How does a Red-Black Tree maintain better balance compared with an ordinary Binary Search Tree?
 10. Compare a **Red-Black Tree** and an **AVL Tree** with respect to balancing and rotation requirements.

References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th Edition, MIT Press, 2022.
2. Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, *Computer Algorithms*, Computer Science Press.
3. Anany Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd Edition, Pearson Education, 2012.
4. Mark Allen Weiss, *Data Structures and Algorithm Analysis*, 3rd Edition, Pearson Education.